Day 13: Correlation Rule Design Exercise

Defining, Manually Testing, and Validating a Brute-Force Detection Rule in Splunk

Field	Details
Intern Name	Amrita Pathak
Date	15 September 2026
Internship Track	SOC (Security Operations Center)
Task Deadline	15 September 2026, 11:59 PM IST

Table of Contents

Table of Contents 2
Executive Summary 3
1. Objective 3
2. Data Declaration 3
3. Tools and Environment 3
4. Key Terms 4
5. Why This Task Matters 4
6. Manual Walkthrough 5
6.1 203.0.113.5 (primary source) 5
6.2 198.51.100.9 (secondary source) 5
6.3 192.168.84.1 (kali, legitimate account) 5
7. Validating the Rule in Splunk 6
7.1 Session Steps 6
7.2 What the Query Does, Line by Line 8
8. Results Summary 9
9. Findings 10
9.1 Manual and automated testing produced an identical result 10
9.2 Checking a threshold against real data changed the decision 10
9.3 The rule correctly separated two attackers of very different volume 10
9.4 This test did not exercise window expiry, and that gap is worth naming 10
10. Conclusion 10

Executive Summary

This task defined a simple brute-force detection rule against the same authentication log already used in two earlier exercises, tested that rule by hand against every event in the file, and then independently validated it using a real Splunk query. The rule defined was: flag any source IP producing three or more failed authentication attempts within a rolling five-minute window.

A threshold of exactly three failures in five minutes was chosen over a higher, seemingly stricter alternative after checking both against the actual data first. A five-failure threshold would have missed one of the two real attacking sources in the dataset entirely, since it only ever produced three failures total, a genuine false negative, not a hypothetical one. The three-failure threshold caught both attacking sources correctly and produced zero false alerts against the one legitimate account in the log.

The manual, event-by-event walkthrough and the automated Splunk query, built using the streamstats command, arrived at the identical result independently: ten events met the alert condition, nine from the primary attacking source and one from the secondary source, with the legitimate account never appearing at all. That agreement between a hand-worked table and a real query is the core evidence this report is built on.

1. Objective

Define a simple detection rule based on repeated failed authentication events, using the same threshold-and-time-window logic real correlation rules rely on, then test that rule manually against a known dataset and validate the result using Splunk.

2. Data Declaration

The data used in this exercise is sample_auth.log, the same synthetic SSH authentication log created for an earlier command-line exercise and later indexed into Splunk for a separate exercise. It is not captured network traffic. Reusing it here for a third time was deliberate: the exact contents, and the correct answer for any query run against it, were already fully known and verified, which made it possible to confirm the correlation rule's manual and automated results against each other with real confidence, rather than trusting either one blindly.

3. Tools and Environment

Item	Detail
Endpoint used	Kali Linux VM (kali), Splunk Enterprise (installed in an earlier exercise)
Data source	sample_auth.log, synthetic SSH authentication log
Detection rule	3 or more failed authentication attempts from the same source IP within a rolling 5-minute window
Validation methods	Manual event-by-event walkthrough; Splunk SPL using streamstats
Saved artifact	Report: Correlation Rule - 3 Failures in 5 Minutes

4. Key Terms

- **Correlation rule**: logic that examines a stream of individual events and decides, based on a pattern across more than one event, whether to raise an alert. This is different from a single-event alert, which reacts to one log line in isolation, a correlation rule specifically requires looking at relationships between multiple events, such as how many came from the same source within a given time.

- **Threshold**: the count of qualifying events required before a rule fires, three failures in this exercise's rule. Set too low, a threshold produces false alarms on ordinary human error; set too high, it risks missing genuine attacks that do not happen to be large enough to cross the bar.

- **Rolling time window**: a continuously moving span of time, five minutes in this rule, used to decide which past events still count toward the current total. An event older than the window length no longer contributes to the count, which is what allows a rule to catch a fast burst of activity without also treating unrelated events from hours earlier as part of the same pattern.

- **True positive / true negative**: a true positive is a case where the rule correctly alerts on something that actually was the pattern being watched for. A true negative is a case where the rule correctly stays silent on something that was not that pattern. Both are needed to trust a rule, a rule that only ever fires proves nothing about whether it can also stay quiet correctly.

- **False negative**: a case where a rule fails to alert on something that genuinely was the pattern being watched for, a missed detection. This is the specific risk that ruled out the higher five-failure threshold considered for this exercise, discussed in Section 7.

- **streamstats (Splunk SPL)**: an SPL command that computes a running, cumulative value as it reads through a stream of events in order, here used to keep a live count of failures per source IP, restricted to a trailing time window, the direct SPL equivalent of the manual running count kept by hand in Section 6.

5. Why This Task Matters

Every earlier exercise in this internship found evidence after something had already happened. This task is different in a way that matters a great deal in real SOC work: it asks for the logic that turns raw events into an alert automatically, before an analyst is even looking. The specific value this builds is worth naming directly:

- **This is literally how real detections are built**: every alert a SIEM ever raises started as someone writing a rule with exactly this shape, a condition, a threshold, and a time window. Writing and testing one directly, even a simple one, is a genuine rehearsal of that real work, not just a simulation of it.

- **A rule is only trustworthy once it has been checked both ways**: confirming a rule fires on real attack patterns proves it can catch something. Confirming it stays silent on legitimate activity proves it will not bury an analyst in noise. This exercise deliberately tested both, using data where the correct answer for each was already known.

- **Threshold choice is a real decision with real consequences, not a formality**: picking a threshold by checking it against real data first, rather than choosing a round number and

assuming it is safe, is what separates a rule that actually works from one that only looks reasonable on paper. Section 7 documents exactly this kind of check.

- **Manual and automated validation together are stronger than either alone**: a hand-worked table proves the logic is understood. A real query proves it can be operationalized. Neither substitutes for the other, and their agreement here is the actual evidence this rule works, not an assumption.

6. Manual Walkthrough

Every failed authentication event in the log was worked through in chronological order, keeping a running count of failures for each source IP, and resetting attention to whichever events remain inside the trailing five-minute window at each point. The rule states changes to ALERT the moment a source's running count reaches three.

6.1 203.0.113.5 (primary source)

Time	Event	Running Count	Rule Status
09:58:02	Failed password for root	1	No alert
09:58:05	Failed password for admin	2	No alert
09:58:09	Failed password for root	3	**ALERT**
09:58:13	Failed password for test	4	**ALERT (active)**
09:58:24	Failed password for root	5	**ALERT (active)**
09:58:47	Failed password for admin	6	**ALERT (active)**
09:58:52	Failed password for guest	7	**ALERT (active)**
09:59:15	Failed password for root	8	**ALERT (active)**
09:59:22	Failed password for admin	9	**ALERT (active)**
09:59:30	Failed password for postgres	10	**ALERT (active)**
09:59:38	Failed password for root	11	**ALERT (active)**

6.2 198.51.100.9 (secondary source)

Time	Event	Running Count	Rule Status
09:58:20	Failed password for admin	1	No alert
09:58:41	Failed password for root	2	No alert
09:59:00	Failed password for root	3	**ALERT**

6.3 192.168.84.1 (kali, legitimate account)

Both authentication events from this address were successful logins, never failures, so this source never enters the failure count at all and never appears in either table above. The rule correctly produces no alert for it under any threshold, since it never has a single failure to begin counting.

One limitation worth stating plainly: the entire log spans only about 96 seconds from first event to last, well under the five-minute window being tested. This exercise fully proves the threshold logic, three failures correctly triggers, two correctly does not, but it does not exercise what happens when the

window expires and older events age out of the count, since nothing in this dataset is old enough for that to occur.

7. Validating the Rule in Splunk

The same logic was then implemented as a real SPL query and run against the identical file, to confirm the manual result independently rather than trusting it on its own. The steps below cover the full session, from opening the terminal to shutting everything down afterward, followed by a point-by-point explanation of what the query itself does.

7.1 Session Steps

- **Step 1 - opened a terminal on Kali and started Splunk:** ran `sudo /opt/splunk/bin/splunk start --accept-license --run-as-root`. Splunk does not start automatically on boot since it was installed and launched manually in an earlier exercise; this command restarts the server process fresh each session. Waited for it to confirm the web interface was ready at http://kali:8000.

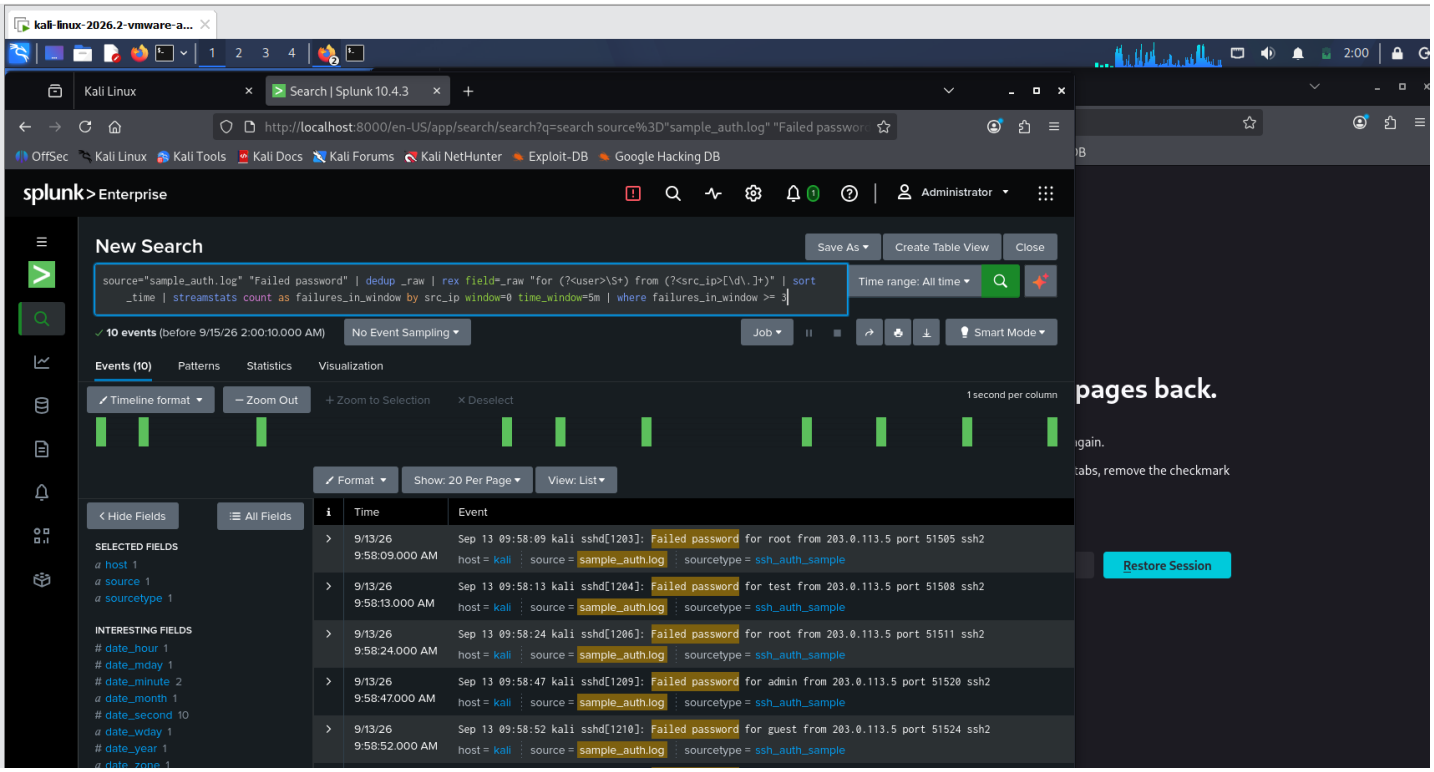
- **Step 2 - opened the Splunk web interface and navigated to the search bar:** opened Firefox to `http://127.0.0.1:8000`, signed in with the existing administrator account, then opened the Search & Reporting app from the Apps menu, which loads directly into the search bar used for every query in this internship's Splunk exercises.

- **Step 3 - typed the correlation rule query and set the time range:** entered the following five-line SPL query into the search bar:

```
source="sample_auth.log" "Failed password" | dedup _raw
| rex field=_raw "for (?<user>\S+) from (?<src_ip>[\d\.]+)"
| sort _time
| streamstats count as failures_in_window by src_ip window=0 time_window=5m
| where failures_in_window >= 3
```

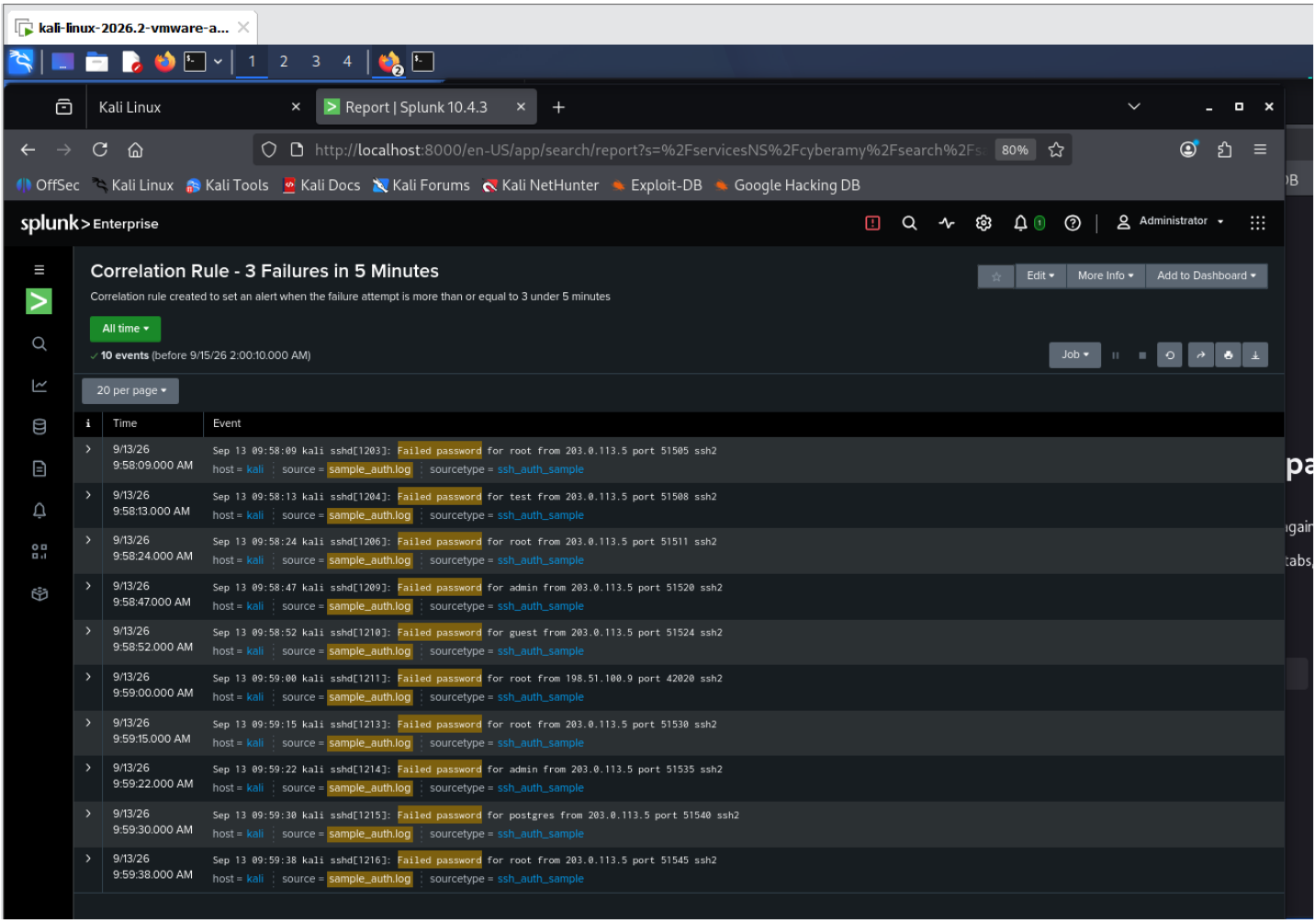
Then changed the time range picker from its default recent window to All Time, since this log's timestamps are fixed on 13 September, and a default range would have silently excluded every event, returning zero results and looking like the query had failed when it had not.

- **Step 4 - reviewed the returned events:** the query returned exactly 10 events. Each one was checked against the source IP and timestamp shown to confirm they matched the same events already identified by hand in Section 6, rather than accepting the count alone as proof.



The streamstats query returning 10 alerting events, matching the manual walkthrough exactly.

- **Step 5 - saved the query as a Report:** used `Save As`, then `Report`, naming it `Correlation Rule - 3 Failures in 5 Minutes`, turning the query into a reusable, named artifact rather than a search that disappears once the browser tab closes.



The saved Report, showing all 10 alerting events as a persistent, reusable rule rather than a temporary search.

- **Step 6 - returned to the terminal and stopped Splunk gracefully:** ran `sudo /opt/splunk/bin/splunk stop`, allowing Splunk to finish writing any pending data to disk before shutting down, rather than cutting it off mid-operation.

- **Step 7 - shut down the Kali VM gracefully:** ran `sudo shutdown now` rather than force-closing the VM window, letting the filesystem unmount cleanly. Both the indexed data and the saved Report persist on disk and were confirmed still present the next time the VM was powered back on.

7.2 What the Query Does, Line by Line

```
source="sample_auth.log" "Failed password" | dedup _raw
| rex field=_raw "for (?<user>\S+) from (?<src_ip>[\d\.]+)"
| sort _time
```

```
| streamstats count as failures_in_window by src_ip window=0 time_window=5m
| where failures_in_window >= 3
```

- `source="sample_auth.log" "Failed password"`: restricts the search to this one file, and to only lines containing the failure message. Everything after this line only ever sees failed attempts.
- `| dedup _raw`: removes exact duplicate lines. This step exists only because of a duplicate upload that occurred during an earlier exercise, not because it is part of the rule's actual logic, on clean data it would not be needed.
- `| rex field=_raw "for (?<user>\S+) from (?<src_ip>[\d\.]+)"`: extracts the username and source IP from each line's raw text. Only src_ip is actually used later in the query, user is captured but never referenced again.
- `| sort _time`: forces events into oldest-first order. This matters more than it looks: Splunk's default order is newest-first, and a running count needs to move forward through time to mean anything, without this line the count would build backwards and the result would be wrong, not just differently ordered.
- `| streamstats count as failures_in_window by src_ip window=0 time_window=5m`: this is the rule itself. For each src_ip separately, keep a running tally of failures within the trailing 5 minutes. window=0 means the count is bounded only by time, not by a fixed number of events.
- `| where failures_in_window >= 3`: keeps only the rows where that rolling count has already reached 3 or more, the exact threshold condition being tested.

The query returned exactly 10 events, matching the manual walkthrough precisely: 9 from 203.0.113.5 (11 total failures minus the first 2 that had not yet crossed the threshold) and 1 from 198.51.100.9 (3 total failures minus the first 2).

8. Results Summary

Item	Value
Detection rule	3+ failed logins from the same source IP within a rolling 5-minute window
Manual walkthrough result	10 alerting events (9 from 203.0.113.5, 1 from 198.51.100.9)
Splunk streamstats result	10 alerting events, identical breakdown
False positives	0 (legitimate account kali never triggered)
False negatives at this threshold	0 (both real attacking sources triggered)
Threshold considered and rejected	5 failures in 5 minutes, would have missed 198.51.100.9 entirely

9. Findings

9.1 Manual and automated testing produced an identical result

The hand-worked table and the Splunk query, built and run independently of each other, agreed on the exact same 10 events with no discrepancy. This is meaningful confirmation, not a coincidence expected by default, two different methods of applying the same logic to the same data landing on an identical answer is strong evidence the rule itself, not just one implementation of it, is correct.

9.2 Checking a threshold against real data changed the decision

A 5-in-5 threshold looked reasonable in the abstract, fewer alerts, a higher bar to clear. Checked against the actual dataset, it would have silently missed a real attacking source that only ever produced 3 failures. This is a concrete demonstration of why a threshold should be validated against real or representative data before being trusted, not chosen by instinct alone.

9.3 The rule correctly separated two attackers of very different volume

203.0.113.5 produced roughly four times as many failures as 198.51.100.9, yet the rule treated both correctly on their own terms, since it evaluates each source independently rather than by total volume across the whole log. A single quieter, secondary source did not get lost underneath a louder primary one.

9.4 This test did not exercise window expiry, and that gap is worth naming

Because every event in this log falls within roughly 96 seconds, the test never exercised what happens once events start aging out of a 5-minute window, only the threshold-crossing behavior. A rule intended for real deployment would need a dataset spanning longer than its window to confirm that expired events are correctly dropped from the running count, not just that enough recent events can trigger it.

10. Conclusion

A brute-force detection rule, 3 or more failed logins from the same source IP within 5 minutes, was defined, deliberately chosen over a stricter alternative only after checking both against real data, and then tested two independent ways: a full manual walkthrough of every event in the log, and a Splunk query using streamstats. Both methods agreed exactly, 10 alerting events, zero false positives against the one legitimate account, zero false negatives against either real attacking source.

The more transferable result here is not the specific number 10, but the process behind it: a threshold is only as good as the data it has been checked against, and a rule is only trustworthy once its logic has been confirmed more than one way. Both of those habits, not just this one rule, are what carry forward into writing real detection logic.